Learning Optimal Policies With Local Observations for Cooperative Multiagent Reinforcement Learning

He Kong, Qianli Xing, Qi Wang, Hechang Chen, Runliang Niu, Zhiyi Duan, *Associate Member, IEEE*, Shiqi Wang, Yi Chang, *Senior Member, IEEE*, and Irwin King, *Fellow, IEEE*

***Abstract*—The cooperative multiagent reinforcement learning (MARL) has been widely used in many practical applications. Despite its success, a fundamental issue arises in MARL that agents face the dilemma of whether to select the best action to maximize rewards or to acquire more information collectively by exploring the novel states/actions due to partial observability. To solve this issue, existing methods merge exploration and exploitation methods. However, these methods are always suboptimal and may lead to failure in finishing tasks. In this article, we theoretically prove the existence of a latent state that can guarantee the optimal individual and global policies. Moreover, we prove that such a latent state can be approximately obtained by local observations. Based on the analysis, we propose a method named unified MARL (UMARL), which is a weighted value function factorization approach unifying exploitation and exploration in one framework. Specifically, we design the agent representation network (ARN) and individual weighting networks (IWNs) to learn agents' unified representations and weights of credit. Moreover, a latent state regularizer (LSR) is designed to encourage agents' representations to approximate the latent state. Extensive experiments show that UMARL can achieve superior performance compared with 12 state-of-the-art methods on *m*-step matrix game, level-based foraging (LBF), StarCraft II, and Google research football (GRF). The source code is available at: https://github.com/CrazyBayes/UMARL**

***Index Terms*—Cooperation, exploration and exploitation, multiagent reinforcement learning (MARL), mutual information (MI) regularizer.**

I. Introduction

RECENTLY, cooperative multiagent reinforcement learning (MARL) has shown advantages in addressing complex practical problems, such as autonomous driving [1], [2], coordination of robot swarms [3], [4], recommendatoins [5], [6], and traffic light systems [7], [8].

Learning optimal policies of cooperative MARL is more challenging than the case of single-agent reinforcement learning, since agents need to consider other agents when selecting actions as they need to achieve a common goal [9], [10]. The agents aim to learn an optimal joint strategy that maximizes the cumulative team return [11], [12]. To improve the coordination among agents, the centralized training with decentralized execution (CTDE) paradigm [13], [14], [15], [16] has attracted much attention, where agents can access the global information during centralized training and execute based on local observations in a decentralized manner.

Nevertheless, even with the CTDE paradigm, the multiagent exploration and exploitation dilemma (MAEED) still exists: agents face the choice of whether to select the best action to maximize rewards or acquire more information collectively by exploring the unexplored/novel states and actions [17], [18], [19], [20]. Due to the issue of partial observability in decentralized execution, agents are constrained to make individual decisions solely based on local state–action histories [21]. Most existing research [22], [23], [24] takes on the challenge of MAEED by simply merging exploration and exploitation methods. For example, QMIX [25] and QPLEX [22] initially formulate the exploitation method to factorize the joint action–value function into individual ones. Afterward, they employ noise-based methods such as ϵ-greedy exploration [26] to hunt for the optimal policy. Alternatively, MAVEN [27] addresses the issue of MAEED and tries to learn optimal policies by augmenting CTDE with committed exploration. However, its exploration and exploitation remain severely compromised due to the persistent information mismatch between local observations and the global state (e.g., partial observability). This may push the MAVEN toward suboptimal policies [28].

The main issue of these methods is the disconnection between exploration and exploitation, which causes poor exploration and suboptimality since the exploration of MARL requires a coordinated manner [27], [29], [30], [31]. As shown in Fig. 1(a), a suboptimal policy for the predator is to attack the prey directly to maximize immediate rewards. As a result, three prey will escape. In contrast, as shown in Fig. 1(b), the optimal policy needs predators to encircle (explore) and attack (exploit) cooperatively: the predators encircle the prey within the blue circle before attacking. In addition to that, studies have shown that too much exploration can also lead

Received 24 December 2024; revised 19 June 2025 and 13 December 2025; accepted 10 March 2026. Date of publication 20 March 2026; date of current version 2 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62206107 and Grant 62406127 and in part by the Scientific and Technological Development Scheme of Jilin Province under Grant 20240101371JC. *(Corresponding author: Qi Wang.)*

He Kong, Qi Wang, Hechang Chen, Runliang Niu, Shiqi Wang, and Yi Chang are with the School of Artificial Intelligence, Engineering Research Center of Knowledge-Driven Human–Machine Intelligence, Ministry of Education, Jilin University, Changchun 130012, China (e-mail: konghe19@mails.jlu.edu.cn; qiwang@jlu.edu.cn; chenhc@jlu.edu.cn; niurl19@mails.jlu.edu.cn; shiqiw23@mails.jlu.edu.cn; yichang@jlu.edu.cn).

Qianli Xing is with the College of Computer Science and Technology, Key Laboratory of Symbolic Computation and Knowledge Engineering, Ministry of Education, Jilin University, Changchun 130012, China (e-mail: qianlixing@jlu.edu.cn).

Zhiyi Duan is with the Department of Computer Science, Inner Mongolia University, Hohhot 010021, China (e-mail: duanzy@imu.edu.cn).

Irwin King is with the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong (e-mail: king@cse.cuhk.edu.hk).

Digital Object Identifier 10.1109/TNNLS.2026.3673692

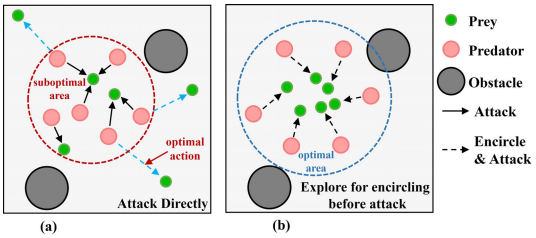

Fig. 1. Motivation. In *Predator-prey*, a suboptimal policy for the predators is to attack the prey directly to maximize immediate rewards as shown in (a). However, to maximize rewards over time, the optimal policy for the predators involves cooperating to explore and encircle the prey within the blue circle before attacking as shown in (b).

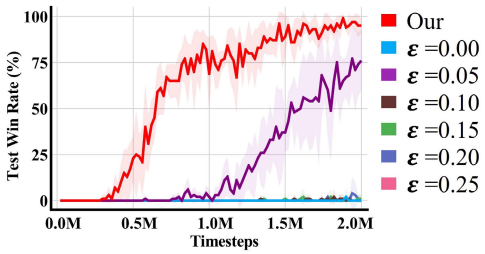

Fig. 2. Learning curve based on QMIX with different ϵ on task *3s_vs_5z*.

to the suboptimal policy convergence [26]. The insight is that the benefit of randomness brought by exploration methods is not fully utilized by the exploitation methods. As a result, the optimal policies are hard to obtain. Fig. 2 shows the learning curves under different values of ϵ in terms of win rate. We observe that QMIX is unable to finish the task *3s_vs_5z* under any exploration setting except when $\epsilon = 0.05$, where it only attains a win rate of 75%. In contrast, the win rate can reach 100% when agents learn to explore efficiently and fully utilize the explored states (as proposed in this article).

However, learning optimal policies for both individual and global policies faces two inevitable challenges. First, with the inconsistency between the local observation during decentralized execution and global information during centralized training, it is challenging to obtain a robust and accurate estimation that unifies multiagent exploration and exploitation. Second, value-based MARL methods involve long-term coordination to solve tasks. To achieve the common goal, agents need to collaboratively discover temporally extended joint strategies to find the optimal policies. Since exploration brings new unexplored state–action spaces, it also poses difficulties in how to utilize these new state–action spaces. Therefore, learning how to explore collaboratively and effectively utilize the explored state–action spaces is challenging.

In this article, to tackle the aforementioned challenges, we first theoretically prove the existence of a latent state unifying exploration and exploitation for guaranteeing optimal individual and global policies. Conditioned on partial observability, we further prove that this latent state can be approximated by a proposed unified representation utilizing local observations. Based on the above analysis, we propose a unified method named UMARL, unified MARL, which is a weighted value function factorization approach with exploration. In concrete, we propose an agent representation network (ARN) that leverages attention mechanisms on local observations to learn a unified representation across agents. Then, each agent is assigned an independent weighting network with its own learned representation as input, and the final joint action–value function is decomposed into a weighted sum of individual action–value functions. As for the exploration, we design a latent state regularizer (LSR) to encourage agents to explore more spaces, where the global state is taken as a supervised signal. The major contributions are summarized as follows.

1) We provide a theoretical analysis to approve the existence of a latent state that can unify exploration and exploitation for value-based MARLs, which can ensure the optimality of both individual and joint policies. Despite the inherent constraints of partial observability, we further provide an approximation of the unified representation mathematically and derive that sufficient exploration can lead to a continuous approximation of the unified representation.
2) We present unified MARL (UMARL), a novel approach for combining weighted value function factorization with exploration. The unified representation across agents is learned via an ARN, which provides the cornerstone for learning optimal policies with local observations. Based on the learned representation, we propose the individual weighting networks (IWNs) for joint action–value function factorization and the LSR for exploration.
3) Extensive experiments conducted on m-step matrix game, level-based foraging (LBF), StarCraft II, and Google research football (GRF) environments comparing with 12 state-of-the-art MARL methods demonstrate the competitive performance of our UMARL.

The structure of this article is organized as follows. Section I provides the related works. Section III shows the prelimaries. The details of UMARL are shown in Section V. Section VI describes the conducted experiments and provides the analysis of the results. Finally, Section VII draws the conclusion and discusses the potential future work.

II. Related Works

In this section, we provide a concise overview of prior works relevant to our proposed method, focusing on two key aspects: value function factorization in MARL and exploration strategies in MARL. This distinction helps to clarify and highlight the unique contributions of our method.

A. Exploitation of Value Function Factorization for MARL

The exploitation of value-based MARL mainly investigates how to choose the actions via action–value functions to maximize the rewards. QMIX [25] learns a monotonic approximation for the joint action–value function Q_{tot}. It achieves this by using a mixing network with nonnegative weights that combine Q_i through a continuous monotonic function. The weights of this function satisfy the individual-global-max (IGM) principle [32], [33]. On the other hand, VDN [34] utilizes the additivity to compute the joint action–value. However, the

assumptions of VDN and QMIX are too strict and only address a fraction of factorizable MARL tasks. QTRAN [32] corrects the discrepancy between the centralized Q_{tot} and the sum of Q_i via a function $V_{jt}(\boldsymbol{\tau})$. QPLEX [22] realizes the full expressive power of value factorization using a duplex dueling network. Later, WQMIX [35] introduces a weighted mechanism into the projection of monotonic value factorization, which allows more emphasis to be placed on better joint actions. PAC [23] leverages the assistive information generated from counterfactual predictions of optimal joint action selection, which enables the explicit assistance to value function factorization through a novel counterfactual loss. ASN [36] characterizes the influence of different actions on other agents using neural networks, leveraging the action semantics between agents to facilitate multiagent value factorization. GoMARL [37] presents an automatic grouping mechanism to group action-values, and these group action–values are used to achieve fine-grained value factorization. These works heavily depend on the global state, which may be burdensome for agents to facilitate the cooperative policy learning. In addition, FACMAC [38] combines the value-based with the actor–critic which learns a single centralized but factored critic to factor the joint-value function into per-agent utilities that are combined via a nonlinear monotonic function. MIXRTs [39] represent explicit decision processes through their root-to-leaf paths and reveal each agent's contribution to the team. GCM [40] leverages a graph-based structure to explicitly model and characterize the complex interaction dynamics within collaborative multiagent relationships. Typically, the above approaches employ the ϵ-greedy policy for exploration. However, it has been proved that such a policy could result in insufficient exploration [27]. In this work, we propose a novel LSR to encourage agents to explore new unobserved states.

B. Exploration in MARL

ϵ-greedy exploration and entropy regularization are the main techniques to improve exploration for MARL methods [26], [29]. Note that entropy the regularization is commonly used in policy-based MARL methods. Thus, we focus on ϵ-greedy exploration in value-based methods. WQMIX investigates the necessity of increased exploration by increasing exploration rate from the 50k and demonstrates that a larger rate of exploration is required to finish the tasks [35]. Unfortunately, increasing the exploration rate in single-agent reinforcement learning is not feasible in the decentralized MARL. In many works, mutual information (MI) is commonly used to assist agents in learning diverse policies [41]. In MAVEN, the MI between the trajectories and latent variables is maximized to learn a diverse set of exploratory behaviors, which can achieve *committed* exploration. To encourage efficient exploration, MAVEN's value-based agents condition their behaviors on the shared latent variable controlled by a hierarchical policy. To coordinate agents' exploration, exploration via information-theoretic influence (EITI) [42] uses the MI to capture the interdependence between the transition dynamics of agents in which agents are encouraged to explore state–action pairs where they can exert influences on other agents. In addition, CMAE [29] investigates states that are worth exploring, where it projects the high-dimensional state space to low-dimensional spaces. Then, CMAE selects goals from restricted spaces and trains the exploration policies to reach the goal. CoopTS [43] explores randomized exploration strategies in parallel MDPs by introducing perturbed-history exploration and Langevin Monte Carlo exploration. LAIES [44], on the other hand, tackles the issue of lazy agents by leveraging a causal graph that captures interactions between agents and the environment, aiming to balance exploration and exploitation more effectively. However, these existing works pay more attention to exploration of MAEED and they fail to take into account efficient exploitation brought by new states/actions, leading to imperfect balance. MAVEN is the most relevant literature to our work. However, MAVEN should sample the latent variable at the start of an episode during execution to achieve exploration, where the exploration of MAVEN heavily depends on this variable. In contrast, we cultivate the sense of exploration for the agents during centralized training and can actively explore without adding additional burden during decentralized execution. Our method also primarily utilizes the explored states or local observations to decompose the global action value into individual ones efficiently.

III. Preliminaries

A. Dec-POMDP

To complete the cooperative multiagent task, a team of agents interacts with the same environment to achieve a common goal [38]. It can be modeled as a decentralized partially observable Markov decision process (Dec-POMDP) [45], which usually can be defined by a tuple $\mathcal{G} = < S, \mathcal{U}, P, r, \Omega, O, \mathcal{N}, \gamma >$. $s \in S$ describes the true state of the environment and is shared by all agents $\mathcal{N} \equiv \{a_1, \ldots, a_n\}$. At each time step, each agent $a_i \in \mathcal{N}$ chooses an action $u_i \in \mathcal{U}$ based on its policy π_i, forming the joint action $\boldsymbol{u} \in \boldsymbol{U} \equiv \mathcal{U}^n$. This causes a transition from s to the next state s' according to the state transition function $P(s'|s, \boldsymbol{u}) : S \times \boldsymbol{U} \times S \to [0, 1]$. $r(s, \boldsymbol{u}) : S \times \boldsymbol{U} \to \mathbb{R}$ is the reward function shared by all agents. In the partially observable setting, each agent a_i receives an individual partial observation $o_i \in \Omega$ from the observation function $O(s, a_i)$. Each agent learns a stochastic policy $\pi_i(u_i|\tau_i)$ based on its action–observation history $\tau_i \in T \equiv (Z \times U)^*$. The joint policy $\boldsymbol{\pi}$ has a joint action–value function: $Q^{\boldsymbol{\pi}}(s_t, \boldsymbol{u}_t) = \mathbb{E}_{s_{t+1:\infty}, \boldsymbol{u}_{t+1:\infty}}[R_t|s_t, \boldsymbol{u}_t]$, where $R_t = \sum_{l=0}^{\infty} \gamma^l r_{t+l}$ is the discounted return and $\gamma \in (0, 1]$ is the discounted factor.

B. CTDE and IGM Principle

CTDE [13], [46], [47], [48] is a commonly used framework in MARL. During training in CTDE, global information including global states of the environment, observations and actions of agents, and policies are available to learn a centralized action–value function. Conversely, agents use their individual policies to make decisions independently based on partial observations during execution. Value-based MARL [34], [35], [49] is one of the main multiagent methods under CTDE framework to solve cooperative multiagent tasks. To learn effective decentralized policies under CTDE, it is essential to ensure that the optimal actions of the joint

action–value function are equivalent to the optimal ones of the individual action–value functions, which is called the IGM principle [22], [32]. An accurate factorization in partially observable MARL problems requires improved representation of the value functions, which is essential for learning optimal decentralized policies [50].

C. Mutual Information

In the information theory, MI is a critical statistical quantity. MI between variables X and Y measures the reduction in entropy of Y by the conditional entropy of Y on X, which can be defined as follows [51], [52]:

$$I(X;Y) = H(Y) - H(X|Y) = \sum_{x\in X}\sum_{y\in Y} P(x,y)\log\frac{P(x,y)}{P(x)P(y)} \tag{1}$$

where $H(\cdot)$ is the entropy function, $H(\cdot|\cdot)$ is the conditional entropy function, x and y are the specific values of variables X and Y. MI quantifies the degree of mutual dependence between two variables, where a higher MI value indicates a stronger relationship between the variables. In addition, the value of MI is nonnegative (i.e., $I(X;Y) \geq 0$ always holds), which can be proved by Jensen's inequality. It has been used in single-agent intrinsically motivated exploration. In MARL, MI is also often introduced to improve the cooperation among agents including multiagent intention sharing and hiding. For example, the MI $I(s, \boldsymbol{u})$ between global state s and joint action $\boldsymbol{u}$ is used to measure the degree of multiagent collaboration [53]. MIR3 [54] introduces MI regularization as a robust constraint, thereby facilitating value function factorization. In this work, MI is utilized to encourage agents explore more spaces for decompositing global action–values more accurate.

IV. Theoretical Analysis

We first prove the existence of the latent state for learning optimal policy. Thus, we have Theorem 1 and provide its proof as follows.

Theorem 1: $s \in S$ describes the true state of the environment, for $\forall a_i \in \mathcal{N}$, $\exists h_i^* \in \mathcal{H}$ (h_i^* denotes optimal latent state and $\mathcal{H}$ is the latent state space) such that the following hold.

1) *Individual Level:* $\forall u_i \in \mathcal{U}_i$ and $Q_i^*(s,u_i) = Q_i^*(h_i^*,u_i)$ holds.
2) *Joint Level:* $\forall \mathbf{u} \in \mathcal{U}$, $Q^*(s,\mathbf{u}) = Q^*(\mathbf{h}^*,\mathbf{u})$ holds, where $\mathbf{h}^* = (h_1^*,\ldots,h_n^*)$.

Proof: Assume a value decomposition structure, where joint action–value function is decomposed into individual action–value functions as follows:

$$Q^*(s,\mathbf{u}) = g\left(Q_1^*(s,u_1),\ldots,Q_n^*(s,u_n)\right)$$

with g being a monotonic mixing function

$$\frac{\partial Q^*}{\partial Q_i^*} \geq 0 \quad \forall i.$$

This structure ensures the IGM property.

For each agent i, define a latent state mapping

$$h_i^* := f_i(s,u_i), \quad h_i \in \mathcal{H}.$$

Formally, define an equivalence relation $\sim_i$ over (s,u_i) such that

$$(s,u_i) \sim_i (\tilde{s},u_i) \iff Q_i^*(s,u_i) = Q_i^*(\tilde{s},u_i).$$

Then, define $h_i^* := [(s,u_i)]_{\sim_i}$ $(f_i(s,u_i) \mapsto [(s,u_i)]_{\sim_i})$ to be the equivalence class of (s,u_i). The construction of this equivalence class is mathematically complete and well-defined, as it satisfies the following.

1) *Reflexivity:* For any s and u_i, it holds that $Q_i^*(s,u_i) = Q_i^*(s,u_i)$ trivially. Therefore, $(s,u_i) \sim_i (s,u_i)$.
2) *Symmetry:* Suppose $(s,u_i) \sim_i (h_i^*,u_i)$, i.e., $Q_i^*(s,u_i) = Q_i^*(h_i^*,u_i)$. By the symmetry of equality, it follows that $Q_i^*(h_i^*,u_i) = Q_i^*(s,u_i)$, hence, $(h_i^*,u_i) \sim_i (s,u_i)$.
3) *Transitivity:* Suppose $(s,u_i) \sim_i (h_i^*,u_i)$ and $(h_i^*,u_i) \sim_i (\tilde{h}_i^*,u_i)$, i.e.,

   $$Q_i^*(s,u_i) = Q_i^*\left(h_i^*,u_i\right) \quad \text{and} \quad Q_i^*\left(h_i^*,u_i\right) = Q_i^*\left(\tilde{h}_i^*,u_i\right).$$

   By transitivity of equality, we obtain $Q_i^*(s,u_i) = Q_i^*(\tilde{h}_i^*,u_i)$, so $(s,u_i) \sim_i (\tilde{h}_i^*,u_i)$.

Hence, the individual-level statement can be proved by construction

$$Q_i^*(s,u_i) = Q_i^*\left(h_i^*,u_i\right).$$

Next, define the joint latent representation

$$\mathbf{h}^* := \left(h_1^*,\ldots,h_n^*\right) = \left(f_1(s,u_1),\ldots,f_n(s,u_n)\right).$$

Then, the global Q value becomes

$$Q^*(s,\mathbf{u}) = g\left(Q_1^*\left(h_1^*,u_1\right),\ldots,Q_n^*\left(h_n^*,u_n\right)\right) = Q^*\left(\mathbf{h}^*,\mathbf{u}\right)$$

due to the monotonic value decomposition structure of g.

Thus, both the individual-level and joint-level conditions are satisfied via the construction of latent states h_i^* from (s,u_i). Thus, we complete the proof. □

Theorem 1 employs the global state s in both the individual-level and joint-level conditions to prove the existence of a latent state h. While this theorem assumes full observability, agents in a partially observable setting lack direct access to s. Consequently, each agent must balance the tradeoff between choosing the action that maximizes the immediate reward and exploring new states and actions to gather information. Under these constraints, the optimal latent state h^* captures the ideal balance between exploration and exploitation.

In MARL, the local observation o_i is determined by the observation function $O(s,u_i)$, we then propose that we can learn an approximation of the proposed latent state based on these observations. In the CTDE paradigm, a discrepancy inherently exists between the global state and local observations due to their differing information scopes. Nevertheless, the IGM principle ensures consistency between the joint action–value function and local action–value functions. Similarly, although h_i^* is derived from the global state and h_i is inferred from the individual agent's local observations, the IGM principle can still preserve their consistency. For exploration, there will be new-unexplored o_i. Thus, with the exploration ability of each agent being more

and more sufficient, h_i is constantly approaching h_i^*. As agent i gradually learns an optimal policy that balances exploration and exploitation, it can decide when to explore purposefully rather than randomly and when to exploit. Moreover, the time spent exploring new areas will gradually decrease. Based on this observation, we propose the following assumption.

Assumption 1: As exploration proceeds, the agent's policy becomes increasingly stable and converges asymptotically to the optimal policy.

It is important to note that Assumption 1 does not simply require the policy to become stable during training. Although such stability is common in many MARL algorithms, it does not ensure that the learned representations converge to the latent optimal states required by our theory. In typical methods, exploration and representation learning are only loosely coupled, so stable policies may still arise from incomplete representations. Assumption 1 instead requires that exploration continuously provides informative observations that drive the unified representation h_i toward its latent optimal state h_i^*, a property not generally guaranteed by standard exploration schemes. We have Theorem 2 and provide its proof as follows.

Theorem 2: Each agent has its local observation o_i, based on these local observations, agent i ($i \in \{1, 2, \ldots, n\}$) can learn a unified representation h_i, i.e., $h_i = f_i(o_1, \ldots, o_n)$. h_i is an approximation of h_i^*, and the difference between h_i and h_i^* is v_i; thus, $h_i^* = h_i + v_i$. When the exploration ability of agent i is becoming more and more sufficient (i.e., the exploration and exploitation is becoming optimal), h_i is constantly approaching h_i^*.

Proof: According to Theorem 1, it follows that both $Q_i^*(s, u_i) = Q_i^*(h_i^*, u_i)$ and $Q^*(s, \boldsymbol{u}) = Q^*(\boldsymbol{h}^*, \boldsymbol{u})$ hold. Without loss of generality, we assume that v_i is a Gaussian noise, i.e., $v_i \sim N(0, \sigma^2, I)$, where σ^2 is the variance and I denotes the identity matrix. The discrepancy is modeled as Gaussian noise $\mathcal{N}(0, \sigma^2\mathbf{I})$ since the uncertainty induced by exploratory behavior during policy learning naturally reflects the agent's exploration ability. According to the Gaussian noise, we should prove

$$Q^*\left(h_i^*, u_i\right) \approx Q_i^*\left(h_i, u_i\right)$$

where $h_i^* = h_i + v_i$.

According to the Taylor expansion, we have

$$Q^*\left(h_i^*, u_i\right) \approx Q_i^*\left(h_i, u_i\right) + \nabla Q^*\left(h_i, u_i\right)^T v_i + \frac{1}{2}\nabla^2 Q^*\left(h_i, u_i\right) v_i$$

where $\nabla Q^*(h_i, u_i)$ is the gradient of $Q_i^*(h_i, u_i)$ with respect to h_i, $\nabla^2 Q^*(h_i, u_i)$ is the Hessian matrix of $Q_i^*(h_i, u_i)$ with respect to h_i.

Because v_i is an Gaussian noise, it is clear that

$$\begin{aligned} E\left[v_i\right] &= 0 \\ E\left[v_i^T v_i\right] &= \sigma^2 I \\ E\left[v_i^T \nabla Q_i^*\left(h_i, u_i\right)\right] &= 0. \end{aligned}$$

Therefore, we have

$$\begin{aligned} E\left[Q_i^*\left(h_i^*, u_i\right)\right] &\approx E\left[Q_i^*\left(h_i, u_i\right)\right] + E\left[\frac{1}{2} v_i^T \nabla^2 Q_i^*\left(h_i, u_i\right) v_i\right] \\ &= E\left[Q_i^*\left(h_i, u_i\right)\right] + \frac{1}{2}\sigma^2 tr\left(\nabla^2 Q_i^*\left(h_i, u_i\right)\right) \end{aligned}$$

where $tr(\cdot)$ is the trace of a matrix. When $Q_i^*(h_i, u_i)$ is a smooth function, the trace of the above Hessian matrix is a finite constant, denoted by C_i. Therefore, we have

$$E\left[Q_i^*\left(h_i^*, u_i\right)\right] = E\left[Q_i^*\left(h_i, u_i\right)\right] + \frac{1}{2}\sigma^2 C_i.$$

Based on Assumption 1, it is concluded that as the exploration ability of agent i becomes increasingly sufficient, the variance σ^2 of v_i will correspondingly decrease. Consequently, the expectations $E[Q_i^*(h_i^*, u_i)]$ and $E[Q_i^*(h_i, u_i)]$ will converge, becoming progressively closer to each other. Note that Gaussian noise with different means can be adjusted through a translation transformation by the Gaussian noise with a mean of 0, and the proof process for other distributions is similar. Thus, we complete the proof. □

Based on the above theorems, the requirement is to make agents' unified representations approximating the latent state with local observations. Accordingly, we propose a method named UMARL, which is a weighted value function factorization approach with exploration. The UMARL contains three core parts: ARN, IWNs, and LSR. The framework is shown in Fig. 3.

V. Methodology

To jointly address the fundamental tradeoff between exploration and exploitation in MARL, we introduce a unified framework grounded in a learnable latent state representation (see Fig. 3). This latent state is inferred from local observations via an ARN, and serves as a shared information bottleneck to support two complementary objectives: 1) adaptive credit assignment, achieved through an IWN and 2) efficient exploration, guided by a LSR that maximizes a lower bound on the conditional MI. Unlike prior approaches that handle exploitation and exploration in isolation, our method integrates them into a cohesive latent representation, thereby enabling more stable training dynamics and improving interagent coordination.

A. Agent Representation Network

Under the CTDE framework, we design the ARN to learn the unified representation by using cooperative agents' local observation rather than the global state for value function decomposition. We model the multiagent system as a soft fully-connected graph where each node denotes each agent, and the edge between agents denotes the visibility. In particular, we propose the frequency-weighted matrix, denoted by M^{FW}, to represent the strength of visibility. During decentralized execution, agents possess a restricted field of view. Consequently, when an agent detects the presence of another, the likelihood of cooperation increases. This suggests that M^{FW} can be interpreted as the weight of the connections in the network of agents. M^{FW} is defined as follows.

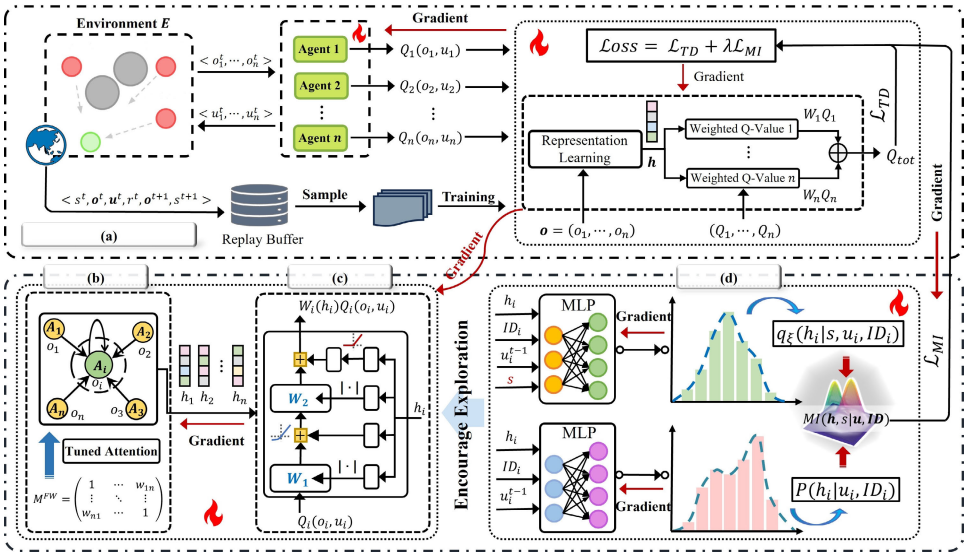

Fig. 3. Architecture of UMARL under the CTDE framework including (a) UMARL framework, (b) ARN, (c) IWN, and (d) LSR. denotes that the parameters need to be trained.

Definition 1 (Frequency-Weighted Matrix): For a batch b sampled from replay buffer D, the symmetric matrix M^{FW} with n agents that counts the number of mutual observations between agents in this batch as a proportion of all observations is called the frequency-weighted matrix

$$M^{FW} = (1, \ldots, w_{1n}, \ldots, w_{n1}, \ldots, 1)$$

where w_{ij} denotes the weight from ith agent to jth agent and $w_{ij} = w_{ji}$. w_{ij} is calculated as follows:

$$w_{ij} = \frac{\sum_{|B|} \operatorname{sgn}\left(f\left(pos_i, pos_j\right)\right)}{|B|}$$

where sgn is the Sign function determined by the Euclidean distance function f indicating the visibility between two agents. pos_i is the position of agent i. $\operatorname{sgn}(f(pos_i, pos_j)) = 1$ if the distance is smaller than the limited vision, and $\operatorname{sgn}(f(pos_i, pos_j)) = 0$ otherwise.

By incorporating the concept of the agent graph, agents' representations can effectively capture the visibility inherent in decentralized execution. To learn the unified representation, the attention aggregation is introduced. We take the local observations of agents as the node features, where $h_i = o_i, i \in \{1, \ldots, n\}$, initially. The coefficient from jth agent to ith agent is calculated as follows:

$$\alpha_{ij} = \frac{\exp\left(LeakyReLU\left(\boldsymbol{Att}\left[\boldsymbol{W_G} h_i \| \boldsymbol{W_G} h_j\right]\right)\right)}{\sum_{m \in \mathcal{N}} \exp\left(LeakyReLU\left(\boldsymbol{Att}\left[\boldsymbol{W_G} h_i \| \boldsymbol{W_G} h_m\right]\right)\right)} \tag{2}$$

where $\boldsymbol{Att}$ denotes the attention mechanism, $\boldsymbol{W_G}$ is the learnable linear transformation and $\|$ is the concatenation operation. To stabilize the learning process, multihead attention [55] is utilized. Then, with the M^{FW}, the unified representation in each convolution is calculated as follows:

$$\boldsymbol{h_i} = \Big\|_{k=1}^{K} \sigma\left(\sum_{j \in \mathcal{N}} w_{ij} \alpha_{ij}^{k} \boldsymbol{W_G^k} h_j\right) \tag{3}$$

where K is the number of attention head and σ denotes the $Sigmoid$ activation function. The final unified representation denoted by $\boldsymbol{h_i}$ is obtained by concatenating the output in each convolution operation. Note that we update the M^{FW} over batches during training because the soft links among agents vary over time dynamically. To utilize the historical links, the M_t^{FW} at training time t is updated as the average of its last value at training time $t-1$ and the value of M^{FW} counted by the current batch as follows:

$$M_t^{FW} = \frac{M_{t-1}^{FW} + M^{FW}}{2}. \tag{4}$$

UMARL leverages all historical link information to iteratively refine and generate improved agent representations.

B. Individual Weighting Networks

In this section, we decompose the joint action–value Q_{tot} in the manner of the weighted sum of individual Q_i in which the weights of agents reflect corresponding contributions to the whole team. Accordingly, we introduce IWNs, which are designed to assess how much each agent contributes to the team's overall success, using the QMIX framework as a basis.

Specifically, each agent has its own IWN, which takes its unified representation as input and outputs the weighted Q_i instead of using a single mixing network to calculate Q_{tot}. The global state is commonly used in existing works to learn credit assignment weights. However, it is challenging to accurately assign credit to individual agents using a mixing network, since the global state is shared among all agents and may overlook the specific situations faced by each agent. On the contrary, as the unified representation is oriented to the execution, it is better to use the unified representation rather than a global state to deduce an agent's contribution. Meanwhile, IWN guarantees the IGM principle. Formally, we decompose the joint action–value function into

the weighted sum of individual action–value functions, as follows:

$$Q_{tot}(\boldsymbol{\tau}, \boldsymbol{u}) = \sum_{i=1}^{n} IWN_i(\boldsymbol{h_i})\, Q_i(\boldsymbol{\tau}, u_i). \tag{5}$$

$IWN_i(\boldsymbol{h_i})$ denotes the corresponding weight learned by IWN for the ith agent and $IMW_i(h_i) \geq 0$ holds. This nonnegativity in IWM guarantees the IGM principle of value-based methods. Moreover, taking the agent representation rather than the global state as the input of IWN contributes to learning the credit of an agent more accurately. Thus, UMARL can be taken as improving the additivity and monotonicity decomposition via weights to correct for the discrepancy between the sum of $[Q_i]$ and the potential true Q_{tot}. Note that, parameter sharing is utilized among these IWNs. Since each agent has a different unified representation, sharing parameters does not preclude them from learning credits differently, as is appropriate. A deep recurrent q-learning network (DRQN) [56] is utilized to learn the individual Q_i of each agent.

C. Latent State Regularizer

According to Theorem 2, the latent state can be approximated by local observations. The underlying assumption is that the more comprehensive the unified representation is, the better a policy can value-based agent learn. Thus, we add a novel regularizer based on MI [52], which encourages the unified representation h_i to constantly approach h_i^*. Theorem 1 indicates that the global state can be used in the derivation of optimal policies. Therefore, during the training phase, we utilize the global state s as a substitute for the unknown h_i^*. Given that the global state s describes the whole environment at each time step, we introduce the MI conditioned on the action and the index of agent i (ID_i) between the global state and the unified representation as follows:

$$MI(h_i, s|u_i, ID_i) = H(h_i|u_i, ID_i) - H(h_i|s, u_i, ID_i) \tag{6}$$

where $H(\cdot|\cdot)$ is the conditional entropy. Such MI between the global state and learned unified representation measures how related they are. To balance the tradeoff between exploration and exploitation, we implicitly encourage exploration by learning a comprehensive latent state h. Given that the global state s includes all details about the environment, we maximize the (conditional) MI between s and h_i based on the action u_i in order to encourage the agent to choose different actions to explore environments as much as possible.

The conditional entropy of $H(h_i|\cdot)$ given s, u_i, and ID_i is not tractable for nontrivial mappings, which makes directly using MI infeasible, as a consequence, using MI directly is infeasible. Therefore, we introduce a variational distribution $q_\xi(h_i|s, u_i, ID_i)$, which provides a lower bound of (6) as follows:

$$\begin{aligned} &MI(h_i, s \mid u_i, I\ D_i) = H(h_i \mid u_i, I\ D_i) - H(h_i \mid u_i, s, I\ D_i) \\ &\geq \mathbb{E}_{h_i, s, u_i, I\ D_i}\left[-D_{KL}\left(P(h_i \mid u_i, I\ D_i) \,\|\, q_\xi(h_i \mid s, u_i, I\ D_i)\right)\right]. \end{aligned} \tag{7}$$

The lower bound matches the exact MI when $q_\xi(h_i \mid s, u_i, I\ D_i)$ equals $P(h_i \mid s, u_i, I\ D_i)$. The value of MI between the unified representation h_i and global state s indicates how big the state space explored by agent i is. Therefore, the MI objective for maximizing the lower bound is capable of encouraging agents to visit more state space to find the potential unified representation. We take this objective (i.e., MI loss) as a regularizer, adding to the total loss during centralized training where two MLPs are utilized to realize it. The MI loss is defined as follows:

$$\begin{aligned} \mathcal{L}_{MI} &= \mathbb{E}_{\boldsymbol{h}, s, \boldsymbol{u}, \boldsymbol{ID}}\left[D_{KL}\left(P(\boldsymbol{h} \mid \boldsymbol{u}, \boldsymbol{ID}) \,\|\, q_\xi(\boldsymbol{h} \mid s, \boldsymbol{u}, \boldsymbol{ID})\right)\right] \\ &= \mathbb{E}_s \sum_{h_i, u_i, ID_i}\left[D_{KL}\left(P(h_i \mid u_i, ID_i) \,\|\, q_\xi(h_i \mid s, u_i, ID_i)\right)\right]. \end{aligned} \tag{8}$$

L_{MI} ensures that newly encountered observations are iteratively incorporated into the agent's latent representation h_i. Thus, as exploration unfolds, L_{MI} progressively helps agents obtain a more and more comprehensive latent state.

Proof: The proof of (7) is as follows:

$$\begin{aligned} &MI(h_i, s \mid u_i, I\ D_i) \\ &= H(h_i \mid u_i, I\ D_i) - H(h_i \mid u_i, s, I\ D_i) \\ &= \mathbb{E}_{h_i, s, u_i, I\ D_i}\left[\log \frac{P(h_i, s \mid u_i, I\ D_i)}{P(h_i \mid u_i, I\ D_i) P(s \mid u_i, I\ D_i)}\right] \\ &= \mathbb{E}_{h_i, s, u_i, I\ D_i}\left[\log \frac{P(h_i \mid s, u_i, I\ D_i)\, q_\xi(h_i \mid s, u_i, I\ D_i)}{P(h_i \mid u_i, I\ D_i)\, q_\xi(h_i \mid s, u_i, I\ D_i)}\right] \\ &= \mathbb{E}_{h_i, s, u_i, I\ D_i}\left[\log \frac{q_\xi(h_i \mid s, u_i, I\ D_i)}{P(h_i \mid u_i, I\ D_i)}\right] \\ &\quad + D_{KL}\left(P(h_i \mid s, u_i, I\ D_i) \mid q_\xi(h_i \mid s, u_i, ID_i)\right) \\ &\geq \mathbb{E}_{h_i, s, u_i, I\ D_i}\left[\log \frac{q_\xi(h_i \mid s, u_i, I\ D_i)}{P(h_i \mid u_i, I\ D_i)}\right] \\ &= \mathbb{E}_{h_i, s, u_i, I\ D_i}\left[\log q_\xi(h_i \mid s, u_i, I\ D_i)\right] \\ &\quad - \mathbb{E}_{h_i, u_i, I\ D_i}\left[\log P(h_i \mid u_i, I\ D_i)\right] \\ &= \mathbb{E}_{h_i, s, u_i, I\ D_i}\left[\log q_\xi(h_i \mid s, u_i, I\ D_i)\right] + H(h_i \mid u_i, I\ D_i) \\ &= \mathbb{E}_{s, u_i, I\ D_i}\left[\int P(h_i \mid u_i, I\ D_i) \log q_\xi(h_i \mid s, u_i, I\ D_i)\, dh_i\right] \\ &\quad - \mathbb{E}_{s, u_i, I\ D_i}\left[\int P(h_i \mid u_i, I\ D_i) \log P(h_i \mid u_i, I\ D_i)\, dh_i\right] \\ &= \mathbb{E}_{h, s, u_i, I\ D_i}\left[-D_{KL}\left(P(h_i \mid u_i, I\ D_i) \,\|\, q_\xi(h_i \mid s, u_i, I\ D_i)\right)\right]. \end{aligned}$$

□

D. Whole Procedure of UMARL

At each time step, the tuple $(s, \boldsymbol{o}, u_i, \ldots, u_n, s', \boldsymbol{o'}, r)$ is collected and saved in the experience replay buffer D, where s' is the next state and $\boldsymbol{o'}$ is the next local observation. The UMARL has several parts of neural networks including: ARN, IWN, LSR, and DRQN. For simplicity, we denote the parameters of these parts as θ, η, ψ, and ϕ, separately. During centralized training, UMARL is trained end to end via minimizing the following loss:

$$\mathcal{L}(\theta, \eta, \phi, \psi) = \mathcal{L}_{TD}(\theta, \eta, \phi) + \lambda \mathcal{L}_{MI}(\psi) \tag{9}$$

where λ is the adjustable multiplier of LSR that can control the strength of the agent representation approximating the global

state. The LSR is achieved by two MLPs with ReLU activation function. These two MLPs are utilized to approximate the distributions $P(h_i|u_i, ID_i)$ and $q_\xi(h_i|s, u_i, ID_i)$. The dimension of the output of each MLP is set to 16 in our experiments.

$\mathcal{L}_{TD}(\theta, \eta, \phi)$ is the TD-error as in DQN, which is defined as follows:

$$\mathcal{L}_{TD}(\theta, \eta, \phi) = \sum_{i=1}^{|B|} \left[\left(y_i^{tot} - Q_{tot}(\boldsymbol{\tau}, \boldsymbol{u}, s; \theta, \eta, \phi) \right)^2 \right] \tag{10}$$

where B is a batch sampled from D, $y^{tot} = r + \gamma \max_{\boldsymbol{u}'} Q_{tot}(\boldsymbol{\tau}', \boldsymbol{u}', s'; \theta^-, \eta^-, \phi^-)$ and θ^-, η^-, and ϕ^- are the parameters of corresponding target networks, which are periodically copied parameters from θ, η, and ϕ. The pseudo code is given in Algorithm 1.

Algorithm 1 UMARL: Unified MARL

```
Initialize the parameters: θ, η, ψ, φ;
Initialize the target parameters: θ⁻ = θ, η⁻ = η, φ⁻ = φ, ψ⁻ = ψ;
Initialize the experience replay buffer D;
for episode = 1 to M do
    Get the initial state s^0, local observations o_i^0 = O(s^0, a_i^0) for each agent a_i  from environment E;
    for t = 1 to max − step do
        Select a random u_i^t with probability ε otherwise u_i^t = arg max_{u_i^t} Q_i(τ_i^t, u_i^t) for each agent i;
        Execute actions u^t = (u_1^t, ..., u_n^t) and receive reward r^t, next state s^{t+1}, and next local observations o^{t+1};
        Store transition (s^t, o^t, u^t, r^t, s^{t+1}, o^{t+1}) into D;
        Sample a random minibatch B from D;
        Compute M_t^{FW} of B and update M_t^{FW};
        Learn global representation h^t = (h_1^t, ..., h_n^t) according to Eq. (3);
        Learn weight [W_i]_{i=1}^n for [Q_i]_{i=1}^n and compute the final Q_tot according to Eq. (5);
        Learn distributions P(h_i | u_i, I D_i) and q_ξ(h_i | s, u_i, I D_i) via incentive exploration regularizer fit by two MLPs;
        Update θ, η, φ, and ψ by minimizing the loss L(θ, η, δ, φ, ψ) = L_TD(θ, η, φ) + λ_MI L_MI(ψ);
    end for
    Update the target parameters with period I: θ⁻ = θ, η⁻ = η, φ⁻ = φ, ψ⁻ = ψ;
end for
```

Remark: UMARL learns optimal policies with local observation by combining multiagent exploration and exploitation into a unified framework. A natural question arises: how does UMARL achieve this integration? In value-based MARL, an algorithm should decompose the global Q-function into individual Q-functions as accurately as possible, which provides substantial advantages for effective exploration. Conceptually, UMARL proposes a unified representation with local observations to compute the individual Q-functions, embedding exploration within this representation. Such a way differs from existing multiagent exploration methods, which directly utilize newly explored states or local observations. In terms of implementation, the L_{MI} term encourages agent exploration, while the L_{TD} term promotes learning accurate individual action values and weights. Although L_{MI} and L_{TD} are applied to different neural networks, they are intrinsically connected by the proposed representation, which is key to integrating multiagent exploration and exploitation.

VI. Experiments

In this section, we evaluate UMARL in m-step matrix game, LBF, a range of StarCraft II decentralized micromanagement tasks from the SMAC environment, and GRF. The 12 state-of-the-art MARL algorithms, including QMIX, MAVEN, and G2ANET (Central-V+G2ANET) [57], QTRAN (QTRAN_alt),[1] QPLEX, and optimistically weighted QMIX (OWQMIX),[2] FACMAC, GoMARL, PAC, ASN [36], RQN [24], and MIR3 are compared. For UMARL, the RMSProp optimizer ($l_r = 1e-3$) is utilized. The discount factor (γ) for the reward of UMARL is set to 0.95 (0.99 for *3s_vs_4s* and *3s_vs_5s* in SMAC). The learning rate is $5e-4$. For exploration, besides the LSR, we use ϵ-greedy with ϵ annealed linearly from 1.0 to 0.05 over 50k time steps and kept constant afterward. The λ_{MI} of UMARL is $1e-5$. We pause training every 20 000 timesteps and run 20 evaluation episodes with decentralized greedy action selection, and the target networks are updated after every 200 episodes in all experiments. The hyperparameters of other algorithms are adopted by their default implementation of PyMARL. To speed up the learning, all agent networks share the same set of parameters. A one-hot encoded agent *ID* is concatenated to agent observations. All the experiments are constructed on CPU Intel[3] Xeon[3] Gold 5220 and GPU NVIDIA A40 with PyTorch.

In the m-step matrix game, agents must escape the local optimum induced by the ϵ-greedy strategy, while achieving the optimal joint behavior requires sufficient coordinated exploration across multiple steps. In LBF, successful food collection relies on accurately decomposing the joint action–value function into individual action–value functions, which poses a nontrivial credit assignment challenge. In SMAC and GRF, the observation spaces and action sets become significantly more high-dimensional and structured. As a result, agents must learn to explore and exploit in a more adaptive and coordinated manner to handle the increased complexity of partial observability, dynamic interactions, and multiagent cooperation. We evaluate the exploration effectiveness of UMARL using the m-step matrix game benchmark and assess its exploitation capability on the LBF benchmark. Subsequently, we report its performance on SMAC and GRF, demonstrating that UMARL achieves a superior balance between exploration and exploitation and delivers stronger comprehensive results. To sum up, our UMARL shows competitive or better performance than 12 existing SOTA methods with the help of a designed unified exploration and exploitation framework.

[1]There are two versions of QTRAN, including QTRAN_base and QTRAN_alt, we choose QTRAN_alt to be a baseline because it shows a generally better performance than QTRAN_base.

[2]There are two versions of WQMIX, including CWQMIX and OWQMIX; we choose OWQMIX to be a baseline as it shows a generally better performance than CWQMIX.

[3]Registered trademark.

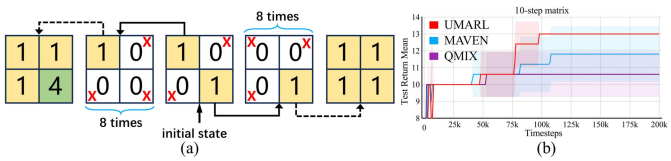

Fig. 4. (a) m-step matrix game for $m = 10$ case. (b) Learning curves of mean returns.

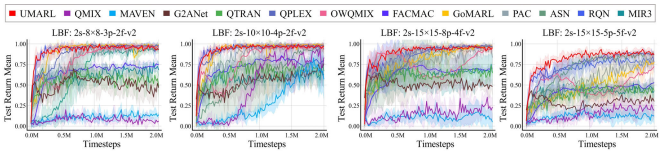

Fig. 5. Learning curves of mean returns on LBF with different settings.

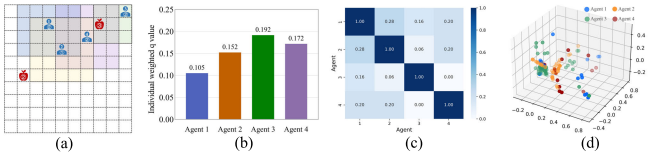

Fig. 6. Case study on LBF (2s-10x10-4p-2f-v2) task.(a) A test frame. (b) Individual weighted q values. (c) Frequency-weighted matrix. (d) Projection of unified representations.

A. m-Step Matrix Game for Exploration

To demonstrate that UMARL can indeed explore the optimal policies, we use a simple m-step matrix game [27], which can also test how nonmonotonicity and exploration interact. At the beginning, the initial state is nonmonotonic, zero rewards lead to termination, and the differentiating states are located at the terminal ends. There will be $m - 2$ intermediate states. As shown in Fig. 4(a), it illustrates the 10-step matrix game. The optimal policy lies in taking the top left joint action and eventually taking the bottom right action, which leads to an optimal payoff $m + 3$. It will become gradually difficult to learn the optimal policy using ϵ-greedy exploration, and enough exploration is necessary.

Fig. 4(b) shows the learning curves of the mean returns of UMARL, MAVEN, and QMIX over five random seeds.[4] We can observe that QMIX falls into the suboptimal, yielding returns near 10, as it struggles with nonmonotonic initial states and ϵ-greedy exploration fails to learn optimal policies effectively. MAVEN achieves superior performance compared to QMIX, yet it is unable to learn the optimal policy within 200k steps. The reason may be that MAVEN's *committed* exploration allows for the discovery of new states or action-state pairs, but the constraints of its mixing network still hinder the exploitation of new states. The UMARL explores the optimal policy with payoff 13, where agents take the top left joint action initially and eventually take the bottom right action. The superior performance of UMARL also illustrates that unifying exploration and exploitation to learn the optimal policy with local observations is a good solution to solve the nonmonotonicity and perform exploration.

B. Performance on LBF

To validate the exploitation ability of our UMARL, we conduct experiments on the LBF benchmark. LBF (LBF version 2) [58] is a partially observable grid world game, where each agent is assigned a level and collects food by cooperating with other agents if needed. Food is randomly scattered, and each also has a level of its own. The collection of food is successful only if the sum of the levels of agents involved in loading is equal to or higher than the level of the food, which can be used to validate the exploitation ability of different value factorization networks. Agents are awarded points equal to the level of the food they helped collect, divided by their contribution (their level). Agents have a restricted vision with a range l, and they need to collect m food.

Fig. 5 shows the learning curves in the form of mean $\pm$ standard deviation with different settings. For example, in *LBF: 2s*-8 $\times$ 8.3*p-2f-v2*, there are three agents and two food in an 8 $\times$ 8 grid world. Agents have a restricted vision with a range of 2. We find that UMARL achieves the task under different settings with the fastest convergence and highest rewards in most steps. This advantage is more obvious in *LBF: 2s*-15 $\times$ 15.5*p-5f-v2*. Such results suggest that decomposing the joint action–value function with the learned unified representation rather than the global state can help to produce better performance. QPLEX and RQN can achieve comparable performance, but QPLEX exhibits slower convergence in all settings while RQN becomes ineffective when the grid world is 8 $\times$ 8. It illustrates that noise-based exploration fails to learn to coordinate quickly. In contrast, UMARL continuously stimulates agents to explore the space, contributing to the fastest convergence. The newly explored local observations can be fully utilized by UMARL to learn optimal policies. In addition, this faster learning convergence also demonstrates the high sample efficiency of UMARL, as more spaces provide better samples to utilize.

1) Visualization for Exploitation: To investigate how UMARL achieves accurate value factorization (exploitation issue), we provide a case study in the LBF (2s-10 $\times$ 10.4p-2f) environment. Fig. 6(a) shows a test frame, where Agents 3 and 4 should cooperate to collect the food with level 3. Fig. 6(b) shows the distributions of individual weighted q values corresponding to Fig. 6(a). As we can see, Agents 3 and 4 have higher individual weighted q values than those of Agents 1 and 2 at this time step, which indicates that UMARL can effectively measure the importance of agents based on the learned unified representations. Fig. 6(c) shows the frequency-weighted matrix of the batch containing the test frame, as shown in Fig. 6(a). Fig. 6(d) shows the visualization of the unified representations of all agents, where the principal component analysis (PCA) [59] is used to reduce the dimensionality of the original unified representation. Taking Agent 1 as an example, it has

[4]We notice that the results of MAVEN are not inconsistent with its original paper. We think that the difference comes from different number of experiments and statistical methods. MAVEN performs m-step matrix game over 20 random initializations rather than five random initializations in this article. MAVEN shows the results in the form of Median test return, where 2nd and 3rd quartiles are shaded rather than the form of mean $\pm$ standard deviation.

TABLE I

MEAN REWARDS AND MRS ON SMAC. S_M_BANE DENOTES SO_MANY_BANELING. THE BEST RESULTS ARE HIGHLIGHTED IN BOLDFACE

Map	*8m*	*3s_vs_4z*	*5m_vs_6m*	*2s_vs_1sc*	*s_m_bane*	*10m_vs_11m*	*MMM*	*3s_vs_5z*	MR
QMIX	18.809±0.985	15.799±1.165	12.714±1.535	12.476±7.360	18.855±0.594	15.937±1.286	17.651±1.386	14.465±1.406	9.13
MAVEN	17.430±1.286	12.255±2.023	7.860±6.550	13.053±6.782	18.449±0.747	11.734±6.151	15.114±1.940	12.435±0.979	6.13
G2ANet	11.982±6.263	4.257±1.491	7.563±0.276	11.753±2.485	16.248±1.359	9.043±2.602	14.188±1.062	2.107±1.291	1.50
QTRAN	17.839±1.315	11.429±1.406	7.938±0.731	17.955±1.276	17.557±1.039	11.644±1.610	13.627±1.554	8.847±0.996	5.25
QPLEX	19.156±0.340	11.303±0.683	14.002±1.058	17.375±1.151	18.977±0.351	14.242±1.424	17.691±0.587	11.609±1.785	9.50
OWQMIX	17.856±0.697	12.024±1.058	10.117±1.289	16.974±0.619	18.031±0.642	11.532±1.705	15.628±1.198	10.130±2.074	6.38
FACMAC	19.294±0.300	8.407±1.408	8.410±0.442	12.316±2.500	18.396±0.648	14.397±1.224	18.543±0.470	7.145±1.173	6.75
GoMARL	**19.493±0.311**	12.263±1.540	13.782±1.281	15.819±7.688	19.150±0.418	14.931±2.595	18.760±0.596	9.728±2.021	9.00
PAC	18.341±0.823	9.961±2.374	11.699±1.184	17.500±1.245	18.747±0.649	12.529±1.352	17.731±0.753	9.584±1.423	8.88
ASN	19.358±0.466	10.824±1.188	**15.062±1.480**	16.577±2.750	18.888±0.543	13.573±1.840	**18.762±0.531**	9.363±1.865	9.38
RQN	16.609±1.941	10.018±1.119	7.849±0.328	16.959±2.029	17.884±0.805	11.999±1.222	16.459±1.835	7.396±1.007	4.63
MIR3	17.181±2.772	5.248±2.477	4.661±3.316	5.462±4.546	18.336±1.252	8.789±4.236	16.325±2.094	3.731±2.508	2.63
UMARL	19.279±0.475	**17.706±0.739**	13.068±1.896	**18.121±1.256**	**19.337±0.352**	**16.059±1.386**	17.842±0.999	**17.562±1.025**	**11.88**

more opportunities to cooperate with other agents as shown in Fig. 6(c), where the values in the first row/column are larger than those in other rows/columns. In Fig. 6(d), the blue circles denoting Agent 1 are distributed throughout the entire space, which proves that UMARL can encourage agents to explore more actively. To sum up, UMARL is a powerful method to unify multiagent exploration and exploitation for learning cooperative policies with local observations.

C. Performance on StarCraft II Micromanagement

To demonstrate that UMARL effectively balances exploration and exploitation while enhancing performance comprehensively, we then consider the combat scenario of StarCraft II micromanagement tasks (SMAC) as the next benchmark. In this article, all SMAC experiments are conducted on the SC2.4.6; the performance is not always comparable across versions. In SMAC, the allay units are controlled by the reinforcement learning agents, while the enemy units are controlled by the built-in AI-controlled units. The combat can be symmetric (i.e., same units for both parties) or asymmetric. The win rate in the StarCraft II Micromanagement Benchmark is the most straightforward metric for evaluating the effectiveness of MARL methods. However, in reinforcement learning, the reward signal provides a more direct and intuitive measure of the model's training progress and performance quality. Moreover, since the objective of MARL is to maximize the expected return, reward serves as a fundamental metric for evaluation. To provide a comprehensive comparison, we show both the test reward and the test win rate in this article. At each time step, agents will receive a joint reward equal to the total damage dealt to the enemy units. In addition, killing each enemy unit and winning the combat will bring additional bonuses of 10 and 200, respectively. These rewards are all normalized to ensure that the maximum cumulative reward achievable within a single episode is 20. We show the learning curves of UMARL and other baselines in this article, in which each reported test win rate is averaged over 20 test episodes for every 20 000 steps.

1) Mean Reward: Table I shows the mean rewards and mean ranks (MRs) of UMARL and baselines on eight SMAC maps over 5 random seeds. The rewards are in the form of mean ± standard deviation. As we can observe, UMARL performs best on five maps. Considering the MR, UMARL achieves the highest rank, followed by QPLEX and ASN. Note that the higher MR indicates better comprehensive performance. Surprisingly, MAVEN did not perform as well as expected. It verifies that although exploration is indispensable to learn the optimal policy, the core essence of value-based MARLs lies in how to represent and utilize the action–value function. Despite utilizing the monotonicity constraints with a simple network, QMIX achieves the competitive performance. Although some experimental results indicate that UMARL does not achieve the best performance in all scenarios, this can be attributed to the following reasons.

1) Firstly, UMARL employs QMIX as its backbone, thereby being affected to some extent by the monotonic constraints inherent in QMIX.
2) On the other hand, agents often struggle to identify states that are truly worth exploring [29]. As a result, our UMARL framework may occasionally explore uninformative or irrelevant regions of the state space, leading to the suboptimal performance in certain scenarios.

In addition, although the differences are not statistically significant, the suboptimal algorithm varies across different maps. For example, QMIX ranks second in *3s_vs_5z*, whereas GoMARL performs second best in *so_many_baneling*. To conclude, combining the value function decomposition with exploration via a unified representation is effective for maximizing rewards.

2) Win Rate: The aim of SMAC is that agents controlled by MARL algorithms win fights with the units controlled by the built-in AI. Thus, the test win rate measures the average win ratio, which is used to demonstrate the performance besides reward. Fig. 7 shows the learning curves of UMARL and other baselines in terms of win rate. As we can see, UMARL achieves comparable or better performance

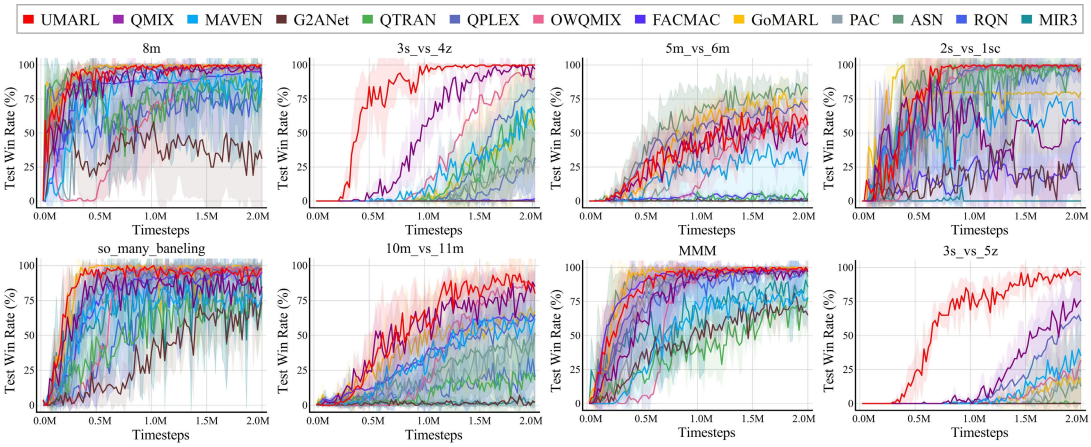

Fig. 7. Learning curves of our UMARL and baselines on SMAC.

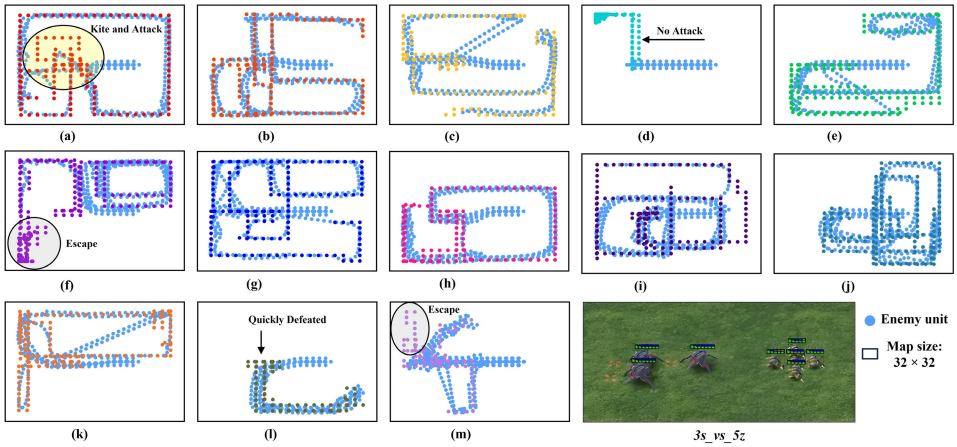

Fig. 8. States visited by different algorithms within 100 steps in *3s_vs_5z*. (a) UMARL. (b) QMIX. (c) MAVEN. (d) G2ANet. (e) QTRAN. (f) QPLEX. (g) OWQMIX. (h) FACMAC. (i) GoMARL. (j) PAC. (k) ASN. (l) RQN. (m) MIR3.

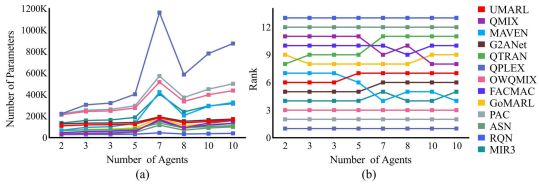

Fig. 9. Complexity analysis in terms of number of parameters. (a) Number of parameters in each map and (b) ranks in each map.

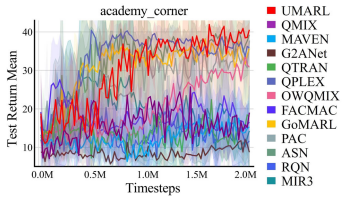

Fig. 10. Learning curves on *academy_corner*.

than other baselines with relatively faster convergence, especially on the maps of *3s_vs_4z* and *3s_vs_5z*. Specifically, G2ANET fails to accomplish tasks on most maps. It proves the necessity of considering visibility among agents, which is utilized by UMARL when aggregating information for exploitation. OWQMIX learns slower than our UMARL on all maps, although it uses the increased ϵ-greedy policy to help agents learn exploratory behaviors. This demonstrates that our UMARL is more intelligent to explore and leverage new states quickly. To sum up, UMARL can explore more spaces and learn optimal policy simultaneously by unifying multiagent exploration and exploitation, making it more powerful to finish tasks.

3) Agents' Ability Brought by Exploration: In this section, we especially focus on *3s_vs_5z*, which is an obvious example to show the agents' ability brought by exploration. *3s_vs_5z* is an asymmetric combat scenario, where *Stalkers* have long-range attack capabilities, while *Zealots* are melee units. Players must leverage the *Stalkers*' long-range advantage and adopt the strategy named *kiting* to avoid damage while gradually depleting enemy units. According to the learning curve for *3s_vs_5z* in Fig. 7, only UMARL successfully completes the task. We present the trajectories of different algorithms within 100 steps in *3s_vs_5z*, as shown in Fig. 8. In the figure, the states visited by MARL agents and built-in AI enemies are represented by points in different colors, and

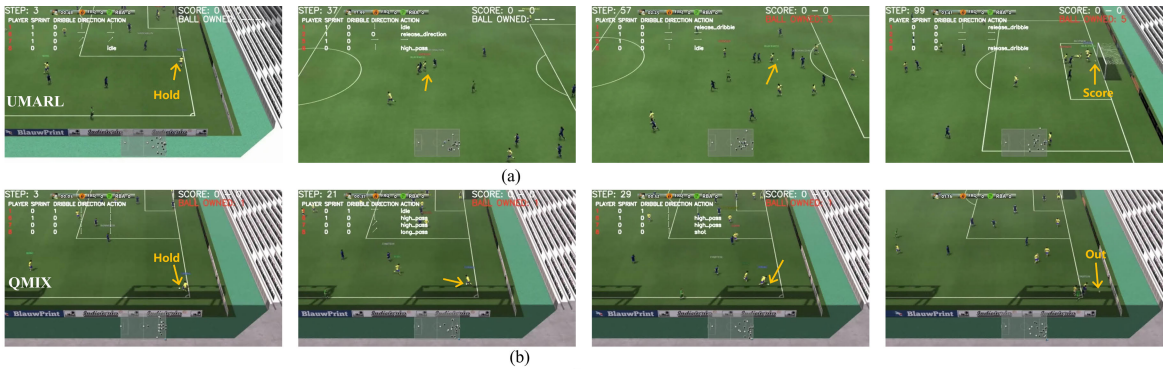

Fig. 11. Example matches between (a) UMARL and (b) QMIX trained agents on *academy_corner*.

the distance between points indicates the difference between states [60]. As depicted in Fig. 8(a), agents trained by UMARL show the *kiting* strategy: they first explore a suitable area to avoid damage (denoted by the yellow area) and then attack the enemies through the proposed value decomposition. In contrast, most other methods, such as QTRAN and PAC, tend to walk alongside the enemies, failing to defeat them. RQN is even defeated by enemies quickly. This demonstrates the superior exploration effectiveness of UMARL. Specifically, agents trained by QPLEX and MIR3 even escape into the corner to avoid being killed. Although MAVEN also uses a latent variable to perform exploration, it neglects to utilize the explored states. In addition, G2ANet explores only a small portion of the entire space and does not exhibit any aggressive behavior. It is unable to complete this task. To sum up, UMARL effectively learns optimal policies with local observation, leading to more intelligent policies and a successful outcome.

4) Complexity Analysis: To illustrate the scaling performance, we conduct a complexity analysis of our UMARL and the compared baselines in terms of the number of parameters on SMAC. Fig. 9 presents the number of parameters and the ranks in each map, where the maps are arranged in ascending order according to the number of agents. As shown in Fig. 9(a), our UMARL requires only slightly more parameters than some parameter-friendly models such as QMIX and RQN. This is noteworthy because UMARL uses attention heads to learn agent representations, and the LSR is implemented through MLPs to encourage exploration. UMARL also enjoys significant advantages over QPLEX, PAC, OWQMIX, and MIR3. The ranking results in Fig. 9(b) further support our analysis. Overall, our experimental results demonstrate that UMARL achieves the SOTA performance and a better exploration–exploitation tradeoff with only a slight increase in the number of parameters.

D. Performance on GRF

GRF is taken as another benchmark to further show the performance of UMARL comprehensively. GRF is a reinforcement learning environment where agents are trained to play football [61]. The number of actions available to each agent is 19, which includes standard move actions (in eight directions) and the actions representing different ways to kick the ball. In addition, GRF has a larger state space than the previously used scenarios, leading to increased difficulty. Fig. 10 shows the learning curves of all baseline methods on *academy_corner*. It can be observed that UMARL learns slower than QPLEX, GoMARL, and ASN at the beginning, yet surpasses all other baselines finally. This can be explained as follows. Although UMARL demonstrates improvements over SOTA methods, we adopt QMIX's mixing network as the IWN, which may limit the representation expressiveness of our value function classes. Also, we propose the regularizer to explore more spaces, which unavoidably leads the agents to explore some useless spaces. As a result, UMARL can achieve better performance with a larger exploration space. Unavoidably, it needs additional time to explore the optimal policies when facing a large action space. In addition, both QMIX and MAVEN fail to finish this task, which means that the unified representation bridging exploration and exploitation is necessary.

1) Example Matches: Fig. 11 shows the example matches in *academy_corner*, in which the classical QMIX is compared. Initially, the ball is held by the reinforcement learning agent while QMIX receives the reward. However, with time going by, agents helped by UMARL can cooperate to score and win the game eventually [as shown in Fig. 11(a)]. In contrast, QMIX exhibits limited cooperative behavior, and the agent quickly kicks the ball out of bounds and loses the game. The reason may lie in a lack of exploration to find distant teammates. Surprisingly, as shown in the top row from step 3 to step 37, agents trained by UMARL are able to pass the ball to distant teammates, thereby increasing their chances of winning the game. As a consequence, the agent can advance to the gate at step 57 and finally learn to score at step 99.

E. Ablation Study

To validate that the components of UMARL, including ARN, IWN, and LSR, are necessary and effective, we conduct ablation experiments on *5m_vs_6m* in SMAC and *academy_corner* in GRF. We use UMARL-A, UMARL-I, and UMARL-L to denote the variants where ARN, IWN, and LSR are removed or replaced from UMARL. Note that

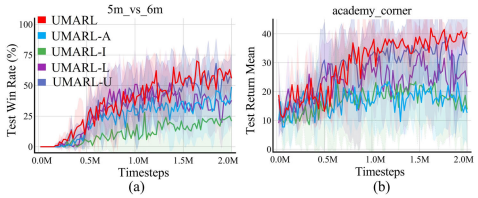

Fig. 12. Ablation studies on (a) *5m_vs_6m* and (b) *academy_corner*.

the input of IWN in UMARL-A is the global state s, and each IWN of UMARL-I is achieved by a two-layer MLP. As shown in Fig. 12, UMARL performs the best among all variants while UMARL-I performs the worst. The failure of UMARL-I demonstrates the critical role of weighting networks in addressing credit assignment. The superiority of UMARL over UMARL-A proves that applying the unified representations universally across all agents, rather than identical global information, is effective for optimal performance. The performance of UMARL-L gradually decreases, which demonstrates the effectiveness of exploring more state spaces. To conclude, ARN and IWN play fundamental roles in UMARL, while LSR can provide a guide signal for exploring more spaces. They play complementary roles, enabling UMARL to have an effective MADRL approach for learning optimal policies.

When learning the unified representation, M^{FW} is updated over batches. To investigate whether the way to update M^{FW} is reasonable and effective, we let UMARL-U denote the variant of our method, which counters over one batch each training time rather than being updated over batches in (4). We can find from Fig. 12 that the original method and UMARL-U perform similarly at the start. However, UMARL-U gradually becomes unstable and finally fails to learn optimal policies. The reason may lie in the fact that, without historical visibility information, UMARL-U will gradually overfit the data collected during decentralized execution, leading to the algorithm instability and even failure.

VII. Conclusion

In this work, we theoretically prove that the optimal policies can be learned by local observations under the CTDE paradigm. Based on theoretical proof, we design a weighted value function factorization approach named UMARL unifying exploitation and exploration in one framework. Specifically, we factorize the joint action–value function into weighted individual action–value functions for exploitation, while encouraging agents to simultaneously explore the optimal policy. We design the ARN and IWN units to guarantee the effectiveness of exploitation, and the LSR unit is further proposed to increase exploration. The proposed unified representation in ARN is the cornerstone for multiagent exploration and exploitation. Extensive experiments are conducted on four benchmarks compared with twelve state-of-the-art MARL algorithms, which prove the competitive performance of our UMARL model. The ablation study shows the effectiveness of each part of UMARL. This article addresses the challenge of learning optimal policies with local observations, specifically within the framework of value-based methods. However, the inherent issue of MAEED remains unresolved in policy-based approaches. In future work, we aim to delve deeper into the complexities of learning optimal policies in policy-based MARL methods, with a particular focus on mitigating the effects of MAEED when relying on localized observations.

References

[1] Y. Cao, W. Yu, W. Ren, and G. Chen, "An overview of recent progress in the study of distributed multi-agent coordination," *IEEE Trans. Ind. Informat.*, vol. 9, no. 1, pp. 427–438, Feb. 2013.

[2] Z. Li, Q. Wang, J. Wang, and Z. He, "A flexible cooperative MARL method for efficient passage of an emergency CAV in mixed traffic," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 8898–8912, Aug. 2024.

[3] M. Hüttenrauch, A. Šošić, and G. Neumann, "Guided deep reinforcement learning for swarm systems," 2017, *arXiv:1709.06011*.

[4] D. Han, C. X. Lu, T. Michalak, and M. Wooldridge, "Multiagent model-based credit assignment for continuous control," in *Proc. 21st Int. Conf. Auto. Agents Multiagent Syst.*, 2022, pp. 571–579.

[5] W. Zhang et al., "RLCharge: Imitative multi-agent spatiotemporal reinforcement learning for electric vehicle charging station recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 6, pp. 6290–6304, Jun. 2023.

[6] Y. Lin et al., "A survey on reinforcement learning for recommender systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 10, pp. 13164–13184, Oct. 2024.

[7] S. Jiang, Y. Huang, M. Jafari, and M. Jalayer, "A distributed multi-agent reinforcement learning with graph decomposition approach for large-scale adaptive traffic signal control," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 14689–14701, Sep. 2022.

[8] Y. Wang, T. Xu, X. Niu, C. Tan, E. Chen, and H. Xiong, "STMARL: A spatio-temporal multi-agent reinforcement learning approach for cooperative traffic light control," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 2228–2242, Jun. 2022.

[9] X. Huang and S. Zhou, "QMNet: Importance-aware message exchange for decentralized multi-agent reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4739–4751, May 2024.

[10] X. Du et al., "Robust multi-agent reinforcement learning via Bayesian distributional value estimation," *Pattern Recognit.*, vol. 145, Jan. 2024, Art. no. 109917.

[11] F. Zhang, Q. Yang, and D. An, "A leader-following paradigm based deep reinforcement learning method for multi-agent cooperation games," *Neural Netw.*, vol. 156, pp. 1–12, Dec. 2022.

[12] S. Ding, W. Du, L. Ding, L. Guo, J. Zhang, and B. An, "Multi-agent dueling Q-learning with mean field and value decomposition," *Pattern Recognit.*, vol. 139, Jul. 2023, Art. no. 109436.

[13] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. 31st Conf. Neural Inf. Process. Syst.*, 2017, pp. 6379–6390.

[14] Z. Xu, Y. Bai, D. Li, B. Zhang, and G. Fan, "SIDE: State inference for partially observable cooperative multi-agent reinforcement learning," in *Proc. Int. Joint Conf. Auto. Agents Multiagent Syst.*, May 2022, pp. 1400–1408.

[15] Z. Li et al., "Coordination as inference in multi-agent reinforcement learning," *Neural Netw.*, vol. 172, Apr. 2024, Art. no. 106101.

[16] H. Kong et al., "Adac: Actor-double-attention-critic for multi-agent cooperation in mixed cooperative-competitive environments," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 7, pp. 9579–9592, Jul. 2025.

[17] M. Taylor, M. Jain, Y. Jin, M. Yokoo, and M. Tambe, "When should there be a 'me' in 'team'?: Distributed multi-agent optimization under uncertainty," in *Proc. 9th Int. Joint Conf. Auto. Agents Multiagent Syst.*, 2010, pp. 109–116.

[18] S. Gronauer and K. Diepold, "Multi-agent deep reinforcement learning: A survey," *Artif. Intell. Rev.*, vol. 55, no. 2, pp. 895–943, Feb. 2022.

[19] L. Stefanos and P. Georgios, "Exploration-exploitation in multi-agent learning: Catastrophe theory meets game theory," *Artif. Intell.*, vol. 304, Mar. 2022, Art. no. 103653.

[20] J.-B. Kim, H.-B. Choi, and Y.-H. Han, "Strangeness-driven exploration in multi-agent reinforcement learning," *Neural Netw.*, vol. 172, Apr. 2024, Art. no. 106149.

[21] C. Yiqun et al., "PTDE: Personalized training with distilled execution for multi-agent reinforcement learning," in *Proc. 33rd Int. Joint Conf. Artif. Intell.*, 2024, pp. 31–39.
[22] J. Wang, Z. Ren, T. Liu, Y. Yu, and C. Zhang, "QPLEX: Duplex dueling multi-agent Q-learning," in *Proc. 8th Int. Conf. Learn. Represent.*, 2020.
[23] H. Zhou, T. Lan, and V. Aggarwal, "PAC: Assisted value factorization with counterfactual predictions in multi-agent reinforcement learning," in *Proc. 36th Conf. Adv. Neural Inf. Process. Syst.*, 2022, pp. 15757–15769.
[24] R. Pina, V. D. Silva, J. Hook, and A. Kondoz, "Residual Q-networks for value function factorizing in multiagent reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 2, pp. 1534–1544, Feb. 2024.
[25] T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson, "QMix: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4295–4304.
[26] J. Hao et al., "Exploration in deep reinforcement learning: From single-agent to multiagent domain," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 7, pp. 8762–8782, Jul. 2024.
[27] A. Mahajan, T. Rashid, M. Samvelyan, and S. Whiteson, "MAVEN: Multi-agent variational exploration," in *Proc. 33rd Conf. Adv. Neural Inf. Process. Syst.*, 2019, pp. 7611–7622.
[28] C. Li et al., "ACE: Cooperative multi-agent Q-learning with bidirectional action-dependency," in *Proc. 37th AAAI Conf. Artif. Intell.*, 2023, pp. 8536–8544.
[29] I.-J. Liu, U. Jain, R. A. Yeh, and A. G. Schwing, "Cooperative exploration for multi-agent deep reinforcement learning," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 6826–6836.
[30] X. Liu and Y. Tan, "Feudal latent space exploration for coordinated multi-agent reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 10, pp. 7775–7783, Oct. 2023.
[31] J. Sun, J. Kou, W. Hou, and Y. Bai, "A multi-agent curiosity reward model for task-oriented dialogue systems," *Pattern Recognit.*, vol. 157, Jan. 2025, Art. no. 110884.
[32] K. Son, D. Kim, W. J. Kang, D. E. Hostallero, and Y. Yi, "QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 5887–5896.
[33] M. Fu, L. Huang, F. Li, H. Qu, and C. Xu, "A fully value distributional deep reinforcement learning framework for multi-agent cooperation," *Neural Netw.*, vol. 184, Apr. 2025, Art. no. 107035.
[34] P. Sunehag, "Value-decomposition detworks for cooperative multi-agent learning based on team reward," in *Proc. 17th Int. Conf. Auto. Agents MultiAgent Syst.*, 2018, pp. 2085–2087.
[35] T. Rashid, G. Farquhar, B. Peng, and S. Whiteson, "Weighted QMIX: Expanding monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. 34th Conf. Adv. Neural Inf. Process. Syst.*, 2020, pp. 10199–10210.
[36] T. Yang et al., "ASN: Action semantics network for multiagent reinforcement learning," *Auto. Agents Multi-Agent Syst.*, vol. 37, no. 2, p. 45, Dec. 2023.
[37] J. Cheng et al., "Automatic grouping for efficient cooperative multi-agent reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. 36*, 2023, pp. 46105–46121.
[38] B. Peng et al., "FACMAC: Factored multi-agent centralised policy gradients," in *Proc. 35th Conf. Neural Inf. Process. Syst.*, 2021, pp. 12208–12221.
[39] Z. Liu, Y. Zhu, Z. Wang, Y. Gao, and C. Chen, "MIXRTs: Toward interpretable multi-agent reinforcement learning via mixing recurrent soft decision trees," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 5, pp. 4090–4107, May 2025.
[40] X. Wu, Y. Zhu, C. Chen, and C. Chen, "GCM: Interpretable multiagent reinforcement learning via graph cooperation modeling," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Nov. 11, 2025, doi: 10.1109/TNNLS.2025.3626000.
[41] H. Kim, J. Kim, Y. Jeong, S. Levine, and H. O. Song, "EMI: Exploration with mutual information," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 1–14.
[42] T. Wang, J. Wang, Y. Wu, and C. Zhang, "Influence-based multi-agent exploration," in *Proc. 8th Int. Conf. Learn. Represent.*, 2020.
[43] H.-L. Hsu, M. Pajic, W. Wang, and P. Xu, "Randomized exploration in cooperative multi-agent reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 74617–74689.
[44] B. Liu, Z. Pu, Y. Pan, J. Yi, Y. Liang, and D. Zhang, "Lazy agents: A new perspective on solving sparse reward problem in multi-agent reinforcement learning," in *Proc. 40th Int. Conf. Mach. Learn.*, 2023, pp. 21937–21950.
[45] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*. Cham, Switzerland: Springer, 2016.
[46] F. A. Oliehoek, M. T. Spaan, and N. Vlassis, "Optimal and approximate Q-value functions for decentralized pomdps," *J. Artif. Intell. Res.*, vol. 32, no. 1, pp. 289–353, May 2008.
[47] Z. Xu et al., "Consensus learning for cooperative multi-agent reinforcement learning," in *Proc. 37th AAAI Conf. Artif. Intell.*, 2023, pp. 11726–11734.
[48] Z. Pu, H. Wang, Z. Liu, J. Yi, and S. Wu, "Attention enhanced reinforcement learning for multi agent cooperation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 8235–8249, Nov. 2023.
[49] J. Castellini, F. A. Oliehoek, R. Savani, and S. Whiteson, "Analysing factorizations of action-value networks for cooperative multi-agent reinforcement learning," *Auto. Agents Multi-Agent Syst.*, vol. 35, no. 2, p. 25, Oct. 2021.
[50] H. Zhou, T. Lan, and V. Aggarwal, "PAC: Assisted value factorisation with counterfactual predictions in multi-agent reinforcement learning," in *Proc. 36th Conf. Adv. Neural Inf. Process. Syst.*, 2022, pp. 15757–15769.
[51] T. M. Cover, *Elements of Information Theory*. Hoboken, NJ, USA: Wiley, 1999.
[52] H. Kong and L. Wang, "Flexible model weighting for one-dependence estimators based on point-wise independence analysis," *Pattern Recognit.*, vol. 139, Jul. 2023, Art. no. 109473.
[53] P. Li et al., "PMIC: Improving multi-agent reinforcement learning with progressive mutual information collaboration," in *Proc. 39th Int. Conf. Mach. Learn.*, 2022, pp. 5845–5854.
[54] S. Li et al., "Robust multi-agent reinforcement learning by mutual information regularization," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 10, pp. 18118–18132, Oct. 2025.
[55] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Conf. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
[56] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable mdps," in *Proc. AAAI Fall Symp. Sequential Decision Making Intell. Agents*, 2015, pp. 29–37.
[57] Y. Liu, W. Wang, Y. Hu, J. Hao, X. Chen, and Y. Gao, "Multi-agent game abstraction via graph attention neural network," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, 2020, pp. 7211–7218.
[58] G. Papoudakis, F. Christianos, L. Schäfer, and S. V. Albrecht, "Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks," 2020, *arXiv:2006.07869*.
[59] S. Wold, K. Esbensen, and P. Geladi, "Principal component analysis," *Chemometrics Intell. Lab. Syst.*, vol. 2, nos. 1–3, pp. 37–52, 1987.
[60] C. Ma, A. Li, Y. Du, H. Dong, and Y. Yang, "Efficient and scalable reinforcement learning for large-scale network control," *Nature Mach. Intell.*, vol. 6, no. 9, pp. 1006–1020, Sep. 2024.
[61] K. Kurach et al., "Google research football: A novel reinforcement learning environment," in *Proc. 34th AAAI Conf. Artif. Intell.*, 2020, pp. 4501–4509.

He Kong received the B.Eng. degree in software engineering from Qufu Normal University, Qufu, China, in 2019, and the M.Eng. degree in computer application technology from Jilin University, Changchun, China, in 2022, where he is currently pursuing the Ph.D. degree in artificial intelligence with the School of Artificial Intelligence.

His research interests include multiagent reinforcement learning, LLM agents, Bayesian networks, and artificial intelligence.

Qianli Xing received the B.E. and M.Sc. degrees in computer science from Jilin University, Changchun, China, in 2014 and 2017, respectively, and the Ph.D. degree in computer science from Macquarie University, Sydney, NSW, Australia, in 2022.

He is currently an Associate Professor with the College of Computer Science and Technology, Jilin University. He has published several papers in high-impact journals and top conferences, including IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, ICML, and AAAI.

Qi Wang received the B.E. and M.Sc. degrees in computer science from Jilin University, Changchun, China, in 2014 and 2017, respectively, and the Ph.D. degree from the Department of Computer Science, Macquarie University, Sydney, NSW, Australia, in 2022.

She is currently an Associate Professor with the School of Artificial Intelligence, Jilin University. She has published several papers in high-impact journals and top conferences, including IJCAI, IEEE ICDM, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, and ICWS. Her main research interests include data mining, graph mining, and LLMs.

Hechang Chen received the Ph.D. degree from the College of Computer Science and Technology, Jilin University (JLU), Changchun, China, in December 2018.

He is currently an Associate Professor with the School of Artificial Intelligence, JLU. He has published more than 60 papers in many top international conferences and journals, such as IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, *ACM Transactions on Knowledge Discovery from Data*, NeurIPS, AAAI, IJCAI, SIGIR, WWW, and ICDE. His current research interests include machine learning, data mining, reinforcement learning, and knowledge engineering.

Runliang Niu received the M.Eng. degree in computer science and technology from Jilin University, Changchun, China, in 2022, where he is currently pursuing the Ph.D. degree in computer science and technology with the School of Artificial Intelligence.

His research interests include reinforcement learning, natural language processing, foundation models, and model interpretation.

Zhiyi Duan (Associate Member, IEEE) received the B.Eng. and Ph.D. degrees from Jilin University, Changchun, China, in 2014 and 2020, respectively.

He is currently a Researcher with Inner Mongolia University Hohhot, Hohhot, China. His research interests include data mining, machine learning, and intelligent education.

Shiqi Wang received the B.Sc. and M.Eng. degrees in computer science and technology from Jilin University, Changchun, China, in 2019 and 2022, respectively, where she is currently pursuing the Ph.D. degree in artificial intelligence with the School of Artificial Intelligence.

Her research interests include natural language processing, machine learning, and multiagent reinforcement learning.

Yi Chang (Senior Member, IEEE) is currently the Dean of the School of Artificial Intelligence, Jilin University, Changchun, China. He became a Chinese National Distinguished Professor in 2017 and the ACM Distinguished Scientist in 2018. He was the Research Director with Yahoo Labs/Research, Sunnyvale, CA, USA, from 2006 to 2016, in charge of search relevance of Yahoo's web search engine and vertical search engines. Before joining academia, he was the Technical Vice President with Huawei Research America, Santa Clara, CA, USA, in charge of knowledge graph, question answering, and vertical search projects. He has authored two books and more than 100 articles in top conferences or journals. His research interests include information retrieval, data mining, machine learning, natural language processing, and artificial intelligence.

Irwin King (Fellow, IEEE) has held prominent leadership roles, including the President of the International Neural Network Society (INNS) and the General Co-Chair of The WebConf 2020, ICONIP 2020, WSDM 2011, RecSys 2013, and ACML 2015. He is currently a Professor with the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong. His research interests span machine learning, social computing, hyperbolic geometry, and multimedia information processing. In these areas, he has authored more than 300 technical publications in leading journals and conferences.